Perbandingan arsitektur aplikasi Campus Notify

Repository konkret dan SOLID dengan struktur layer first

Nama: Sahrul Ramadhani
Kelas: TI 3C

1 Tujuan dan kondisi awal

Laporan ini membandingkan arsitektur berbasis repository konkret dengan hasil refaktor yang menerapkan SOLID dan struktur layer-first. Fitur pengguna tetap sama. Perubahan berfokus pada pembagian tanggung jawab dan arah ketergantungan kode.

Arsitektur awal sudah memakai folder data, pages, providers, dan messaging serta menempatkan akses API dalam repository. Istilah repository konkret digunakan karena presentation langsung memakai kelas implementasinya. Entity masih membaca JSON, sedangkan domain layer dan use case belum dipisahkan.

2 Arti struktur layer first

Layer-first berarti folder utama dikelompokkan berdasarkan tanggung jawab teknis. Login dan notifikasi masing-masing dapat mempunyai file di data, domain, dan presentation. Jadi, folder utama bukan nama fitur.

Layer	Tanggung jawab	Contoh isi
lib/data	Mengambil dan menyimpan data serta menghubungkan SDK.	Dio, model JSON, Firebase, secure storage.
lib/domain	Menentukan bentuk data bisnis, kontrak, dan urutan operasi.	AuthSession, AuthRepository, Login, OpenAnnouncement.
lib/presentation	Menampilkan halaman, mengelola state, dan navigasi.	LoginPage, AuthNotifier, NotificationController, router.

Contoh refaktor: data/auth_repository.dart dipisah menjadi domain/repositories/auth_repository.dart sebagai kontrak dan data/repositories/api_auth_repository.dart sebagai implementasi API. Semua path contoh dalam laporan dihitung dari lib/.

3 Arah dependensi

Presentation memakai kontrak dan use case domain. Data mengimplementasikan kontrak domain. Domain berisi Dart murni dan tidak mengimpor Flutter, Dio, Firebase, atau secure storage.

File presentation/providers/dependencies.dart menjadi tempat memasangkan kontrak dengan implementasinya melalui Riverpod. File ini memang mengenal data layer karena bertugas merangkai dependensi. Halaman dan state lainnya tidak mengimpor adapter data.

4 Perbandingan alur login

Kontrak atau interface menyebut operasi yang wajib disediakan. Repository menyediakan operasi akses data. Use case mengatur satu alur aplikasi, misalnya login.

Repository konkret: AuthNotifier di providers/auth_provider.dart memanggil AuthRepository konkret, menerima sesi, lalu menyimpan access token, refresh token, dan ID pengguna melalui TokenStore. Pengaturan state dan urutan penyimpanan sesi berada dalam notifier yang sama.

SOLID dan layer-first: AuthNotifier di presentation/providers/auth_provider.dart memanggil use case Login. Login di domain/usecases/login.dart menerima kontrak AuthRepository dan SessionStore. Implementasi ApiAuthRepository menghubungi API, sedangkan SecureSessionStore menyimpan token.

Dampaknya: notifier fokus pada loading, berhasil, dan gagal. Urutan autentikasi serta penyimpanan sesi dapat diuji tanpa widget, Firebase, atau penyimpanan perangkat nyata.

Alur awal: halaman → AuthNotifier → repository konkret → API; kemudian notifier → TokenStore. Alur SOLID: halaman → AuthNotifier → Login → kontrak repository dan penyimpanan → implementasi data.

5 Perbandingan entity dan model JSON

Repository konkret: Announcement berada dalam data/inbox_repository.dart dan mempunyai factory fromJson. Representasi data pengumuman serta aturan membaca respons API berada dalam kelas yang sama.

SOLID dan layer-first: domain/entities/announcement.dart hanya menyimpan id, title, body, dan createdAt. data/models/announcement_model.dart membaca JSON dan menyediakan toEntity. Presentation menerima entity domain.

Dampaknya: jika nama field JSON atau format respons API berubah, penyesuaian dipusatkan pada model dan adapter data. Halaman tetap memakai bentuk entity yang sama.

6 Perbandingan membaca pesan dan notifikasi

Repository konkret: messaging/push_service.dart menggabungkan integrasi Firebase, notifikasi lokal, sinkronisasi inbox, badge, dan pemberitahuan perubahan state melalui ChangeNotifier. Halaman detail ikut mengatur pemuatan dan penandaan pesan.

SOLID dan layer-first: FirebaseNotificationRepository berada di data dan menangani SDK serta sinkronisasi. NotificationController di presentation meneruskan state ke UI. Use case OpenAnnouncement memuat detail lalu meminta penyimpanan status baca melalui kontrak.

Dampaknya: kode tampilan berkomunikasi lewat kontrak. Aturan membuka pesan dapat diuji terpisah; pesan hanya ditandai dibaca setelah detail berhasil dimuat. Antrean pesan, suara, dan badge tetap dipertahankan.

7 Perbandingan penerapan SOLID

SOLID adalah lima prinsip pembagian tanggung jawab dan ketergantungan kelas. Struktur layer-first membagi lokasi file, sedangkan SOLID mengatur hubungan antarbagiannya. Penerapannya terlihat pada kontrak, use case, model, dan pemasangan dependensi berikut.

S Single Responsibility

Artinya, satu bagian mempunyai satu tanggung jawab utama. Pada arsitektur awal, entity membaca JSON dan AuthNotifier mengatur state sekaligus penyimpanan sesi. Pada arsitektur SOLID, Announcement menjadi data bisnis, AnnouncementModel membaca JSON, Login mengatur alur autentikasi, dan AuthNotifier mengelola state. Perubahan satu urusan lebih mudah dilacak.

O Open Closed

Artinya, perilaku dapat diperluas melalui implementasi baru dengan sedikit perubahan pada pemakainya. Pada arsitektur awal, provider langsung membuat AuthRepository konkret. Pada arsitektur SOLID, provider bertipe AuthRepository domain dapat dipasangkan dengan ApiAuthRepository atau implementasi lain. Pergantian adapter dilakukan pada dependencies.dart; halaman dan Login tetap memakai kontrak yang sama.

L Liskov Substitution

Artinya, implementasi pengganti tetap memenuhi kontrak yang diharapkan. Pada arsitektur awal, repository dapat diberi Dio untuk pengujian, tetapi kontrak domain belum dipisahkan. Pada arsitektur SOLID, repository palsu mengimplementasikan interface yang sama. Tes Login dan OpenAnnouncement memeriksa hasil sukses serta AppFailure. Pengganti wajib menjaga arti hasil dan kegagalannya.

I Interface Segregation

Artinya, pemakai cukup bergantung pada kontrak yang dibutuhkannya. Pada arsitektur awal, TokenStore memuat sesi serta identitas instalasi dan token FCM, sementara PushService mengoordinasikan banyak urusan. Pada arsitektur SOLID, kontrak AuthRepository, InboxRepository, DeviceRepository, NotificationRepository, SessionStore, dan DeviceIdentityStore dipisahkan. Login hanya menerima auth dan sesi; OpenAnnouncement menerima inbox dan notifikasi.

D Dependency Inversion

Artinya, aturan aplikasi bergantung pada abstraksi, sedangkan teknologi mengisi implementasinya. Pada arsitektur awal, notifier mengimpor repository data konkret. Pada arsitektur SOLID, Login menerima AuthRepository dan SessionStore dari domain. ApiAuthRepository dan SecureSessionStore mengimplementasikan kontrak tersebut, lalu Riverpod menyuntikkannya. Domain tidak perlu mengetahui cara Dio atau secure storage bekerja.

8 Hal yang berubah dan tetap sama

Aspek	Repository konkret	SOLID dan layer-first
Organisasi kode	Data, halaman, provider, dan messaging.	Data, domain, dan presentation.
Aturan login	Di dalam AuthNotifier.	Di use case Login; state tetap di notifier.
Pembacaan JSON	Di kelas entity pada file repository.	Di model data; entity berada di domain.
Notifikasi	PushService juga menjadi ChangeNotifier.	Adapter Firebase dan controller presentation terpisah.
Fungsi pengguna	Login API, inbox, detail, status baca, FCM, badge, suara, topik.	Fungsi yang sama dipertahankan.
Penyimpanan sesi	TokenStore konkret dipanggil langsung.	SessionStore menjadi kontrak; SecureSessionStore menjadi adapter.
Pengujian	Repository dapat diberi Dio pengganti.	Use case diuji melalui interface tanpa infrastruktur nyata.

9 Hasil pemeriksaan

flutter analyze selesai tanpa masalah. Sebanyak 24 tes Flutter lulus, termasuk tes use case sukses dan gagal, refresh token, logout, antrean pesan, serta tiga tes batas dependensi layer. Backend lulus 14 tes dengan 68 assertions. Tes domain memakai repository palsu sehingga aturan aplikasi dapat diuji tanpa server atau Firebase nyata.

Kedua implementasi diuji pada Vivo Android 11 dan berhasil login ke API masing-masing. Token FCM keduanya berbeda. Pesan uji masuk ke aplikasi yang dituju; ketukan membuka detail dan status baca tersimpan. Implementasi SOLID juga menerima FCM setelah proses aplikasi dihentikan. Pesan yang dibaca hilang dari daftar dan badge berkurang. Bukti terdapat pada docs/validasi.md.

10 Manfaat dan konsekuensi

Manfaat utamanya adalah alur bisnis lebih mudah diuji, perubahan respons API dipusatkan pada data layer, dan halaman lebih sedikit mengenal teknologi penyimpanan atau jaringan. Konsekuensinya, jumlah file bertambah dan pengembang perlu memahami hubungan interface, use case, adapter, serta provider.

Arsitektur berbasis repository konkret memisahkan akses API dari halaman, tetapi state masih mengenal implementasi data secara langsung. Arsitektur SOLID dengan struktur layer-first menambah kontrak domain dan use case, lalu memasangkan adapter melalui provider. Perbedaan utamanya ada pada pembagian tanggung jawab dan ketergantungan kode; fungsi pengguna tetap dipertahankan.